



南開大學
Nankai University

计 算 机 学 院

大数据计算及应用实验报告

PageRank 算法设计与实现

姓名：胡进喆 原敬闰 张明昆

学号：2213045 2211771 2211585

专业：计算机科学与技术 物联网工程

指导老师：杨征路

日期：2025 年 4 月 27 日

摘要

目录

1. 前景概要

- 1.1. 背景介绍
- 1.2. 核心思想
- 1.3. 实验要求
- 1.4. 编程语言
- 1.5. 数据集说明

2. PageRank

- 2.1. 算法概述
 - 2.1.1. 核心步骤
 - 2.1.2. 问题修正
 - 2.1.3. 案例分析
- 2.2. 具体实现
 - 2.2.1. 初始化与数据载入
 - 2.2.2. 数据预处理
 - 2.2.3. PageRank计算

3. Block-stripe

- 3.1. 算法概述
 - 3.1.1. 初始版本中的问题
 - 3.1.2. CSR稀疏矩阵格式
 - 3.1.3. 块矩阵优化（分块计算）
 - 3.1.4. 分块矩阵的核心思想
 - 3.1.5. 优化设计
- 3.2. 具体实现

4. 参考文献

1. 前景概要

1.1 背景介绍

随着互联网的飞速发展，全球信息量呈现指数级增长。据统计，截至2023年，全球已有超过20亿个活跃网站和数万亿个网页。在如此庞大的信息海洋中，如何高效、准确地获取所需信息，成为了用户和技术人员共同关注的焦点。传统的搜索技术难以满足用户对速度和精度的要求，信息过载和信息迷航的问题日益凸显。

为了解决这一挑战，Google在20世纪90年代末推出了PageRank算法。这一革命性的网页排名算法基于网页之间的链接关系，评估每个网页的“重要性”或“权威性”，从而提高搜索结果的相关性和质量。PageRank的引入不仅提升了搜索引擎的性能，也在很大程度上推动了互联网信息检索技术的发展。

1.2 核心思想

PageRank算法的核心思想是利用互联网网页之间的链接关系，评估每个网页的重要性或权威性：

- 链接投票机制**：每个网页都被视为一个节点，网页之间的超链接被视为节点之间的边。当一个网页链接到另一个网页时，相当于对其进行了一次“投票”。这些投票用于衡量被链接网页的重要性。
- 权重传递**：投票的权重并非一视同仁。一个网页所赋予的投票权重取决于其自身的重要性（即PageRank值）和出链数量。如果一个高权重的网页链接到某个网页，那么该链接将对目标网页的重要性产生更大的影响。
- 迭代计算**：PageRank值的计算是一个迭代的过程，通过多次重复计算，直到PageRank值收敛，得到每个网页的稳定排名。

可以将整个互联网视为一个巨大的有向图，基于这个视角，我们可以构建一个随机游走模型，也就是一阶马尔可夫链。在这个模型中，我们假设一个虚拟的网页浏览者会随机地、按照等概率地跟随一个页面上的任何一个超链接到另一个页面，并持续这种随机跳转。在长时间内，这种随机跳转的行为会形成一个稳定的模式，即**马尔可夫链的平稳分布**。每个网页的PageRank值，实际上就是在这个平稳分布中的概率。

直观上，一个网页，如果指向该网页的超链接越多，随机跳转到该网页的概率也就越高，该网页的PageRank值就越高，这个网页也就越重要。一个网页，如果指向该网页的PageRank值越高，随机跳转到该网页的概率也就越高，该网页的PageRank值就越高，这个网页也就越重要。PageRank值依赖于网络的拓扑结构，一旦网络的拓扑(连接关系)确定，PageRank值就确定。

1.3 实验要求

本次项目数据集是 Data.txt，实现 PageRank 算法并打印前 100 个节点 ID 及其对应的 PageRank 分数，具体要求如下：

- 核心任务**：输出前 100 个节点及其分数（Res.txt），必须处理 **死胡同节点** 和 **蜘蛛陷阱**，确保算法正确性。
- 强制优化要求**：
 - 内存优化**：必须使用 **块矩阵** 和 **稀疏矩阵** 技术（如 CSR/CSC 格式），确保程序最大内存使用量 **低于 80MB**。
 - 时间性能**：在 **60秒内完成**，不可因内存优化牺牲过多时间效率。
- 实现细节**：迭代至收敛（可设定阈值如 `EPS=1e-8`）；至少报告随机跳跃参数 `damping=0.85` 的结果，可对比其他参数。
- 团队与学术规范**：报告中需明确团队成员贡献，否则默认均分，严禁代码抄袭与学术欺诈，违者零容忍。

1.4 编程语言

经过技术指标和评分维度的综合分析，我们小组最终采用C++实现作为实验的编程平台，以下是C++与Python的对比分析：

评估维度	C++方案优势	Python方案风险
内存控制	手动内存管理+STL容器优化，可精准控制内存碎片	依赖第三方库的内存实现，存在隐性内存开销
执行速度	原生编译优化+OpenMP并行，突破时间限制	解释器性能瓶颈，大规模矩阵运算可能超时
内存优化	可定制化实现块状存储、压缩稀疏矩阵等底层结构	受限于scipy.sparse的预设格式，灵活性较低
跨平台部署	静态编译生成独立可执行文件，兼容性极佳	需处理Python环境依赖，打包存在版本兼容风险
学术展示	展示底层优化细节，易体现技术深度	依赖库函数的黑箱操作，技术细节展示受限

选择C++方案可在满足严苛技术指标的同时，最大限度展示计算系统的优化能力，同时还可避免Python内存泄露陷阱、类型转换开销等较难控制的风险，是较为良好的技术方案。

1.5 数据集说明

本次实验的数据集为 data.txt，其数据格式为每行按照的方式进行存储，表示图中存在从结点 srcNodeID 到结点 dstNodeID 的一条边。我们编写脚本对数据集进行详细的统计分析，得到如下表格：

- **节点统计：**总节点数：9,500 最小节点ID：0 最大节点ID：9999
- **边统计：**总边数：150,000 唯一边数：150,000
- **入度统计：**平均入度：15.7895 最大入度：33.0000 最小入度：2.0000
- **出度统计：**平均出度（有效节点）：17.6471 最大出度：38.0000 最小出度：4.0000
- **特殊节点统计：**死胡同节点数：1,000 (10.53%) 无出度节点数：0 (0%)

该数据集所包含的结点具有较为均衡的入度和出度分布，节点和边的数量都较多，表明图的连接较为密集；所有边都是唯一的，不存在重复边，即无需进行额外的去重工作。

还需注意的是，最小出度为4，说明该数据集不存在与其他节点之间无 out-links的结点，即Spider Traps。而死胡同节点比例较高（约 10.5%），在后续计算 PageRank 时需要考虑死胡同节点的影响。

2. PAGERANK

基本的PageRank 算法通过模拟随机游走在图结构中的行为来评估节点的重要性。首先，算法将每个节点的初始 PageRank 值设为均值 $1/N$ ；接下来，根据节点间的链接关系计算每个节点的出度，并在迭代过程中更新每个节点的 PageRank 值，直到达到预设的收敛条件。为了处理死胡同节点和蜘蛛陷阱问题，算法引入了阻尼因子 $\beta \in (0,1)$ ，通过随机跳转确保每个节点都有非零的 PageRank 值。最终，通过归一化处理，所有节点的 PageRank 值之和为 1。这种机制不仅解决了等级泄露和等级沉没的问题，还使算法能够有效地处理复杂的网络结构，如处理闭环结构和权重集中现象。

2.1 算法概述

2.1.1 核心步骤

初始化阶段：设网络包含 N 个节点，初始化各节点 PageRank 值为均匀分布，即在无先验信息时，假设所有节点具有同等重要性：

$$PR^{(0)}(v_i) = \frac{1}{N}, \quad \forall v_i \in V \quad (1)$$

- 其中 V 为节点集合， $N=|V|$ 。

出度计算：根据网页间的链接关系，计算每个网页的出度（即从该网页指向其他网页的链接数量）。出度信息是后续迭代计算的基础，其定义为 $L(v_j) = \sum_{k=1}^N \mathbf{1}_{\{(v_j \rightarrow v_k)\}}$ ，需严格排除自环边对出度的影响。

迭代计算：通过计算所有节点的新的 r 值与旧的页面节点的 r 值之间的差值，当小于一个预先设定的阈值时算法收敛，迭代停止：

$$PR^{(k+1)}(v_i) = \sum_{v_j \in M(v_i)} \frac{PR^{(k)}(v_j)}{L(v_j)}, \quad k \geq 0 \quad (2)$$

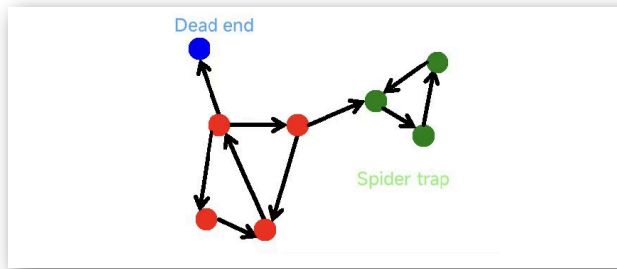
- 其中 $M(v_i)$ 为指向 v_i 的节点集合。

归一化处理：最终通过归一化确保概率分布性：

$$PR_{\text{norm}}(v_i) = \frac{PR(v_i)}{\sum_{j=1}^N PR(v_j)} \quad (3)$$

2.1.2 问题修正

PageRank算法在建模过程中需严谨处理两个关键问题：其一，网页出度的计算需包含自环链接的影响，确保转移概率矩阵的完备性；其二，针对孤立节点引发的吸收态问题，采用随机跳转机制引入遍历性保障。



dead ends (如上图蓝色节点)是指没有任何出链的节点，这种节点破坏了随机游走以及一阶马尔可夫链的假设，最终导致计算出来的PageRank全为0。为了解决上述问题，引入虚拟全局链接，修正出度为：

$$\tilde{L}(v_j) = \begin{cases} L(v_j), & L(v_j) > 0 \\ N, & L(v_j) = 0 \end{cases} \quad (4)$$

该公式将无出链的 Dead Ends 节点出度修正为网络总节点数 N^* ，使其转移概率均匀分配至所有节点（即 $1/N$ ），从而维持转移矩阵的随机性，保障马尔可夫链稳态解的存在性。

spider trap是指在网络图中存在的一个区域，所有的出链都指向该区域内的其他页面，而不会指向外部页面。此类结构会导致：

1. **权重聚集**：随机游走者一旦进入陷阱，无法脱离，闭环内节点的 PageRank 值无限趋近于 1。
2. **收敛失效**：转移矩阵 M 的稳态解仅集中于陷阱内部，破坏全局重要性分布。

因此，我们引入阻尼因子 $\beta \in (0,1)$ ， β 表示用户依链接跳转的概率， $1-\beta$ 为随机跳转概率，这种修正将以 $1-\beta$ 的概率强制用户跳转至任意节点，从而有效解决了等级泄露（Rank Leak）和等级沉没（Rank Sink）的收敛障碍，进而打破闭环内的无限循环。修正后的 PageRank 方程为：

$$PR(v_i) = \beta \left(\sum_{v_j \in M(v_i)} \frac{PR(v_j)}{\tilde{L}(v_j)} \right) + \frac{1-\beta}{N} \quad (5)$$

二项称为平滑项，由于采用平滑项，所有结点的 PageRank 值都不会为 0。至此，我们又可以重新迭代网页的权重计算了，数学定理已证明，最终 PageRank 随机能够收敛。

2.1.3 案例分析

为了更好的说明，我们给出迭代示例，左图为有向图，右图为各节点信息表格：



考虑上图所示的四节点网络 $V=\{A,B,C,D\}$ ，边集合为 $\{A \rightarrow B, A \rightarrow C, B \rightarrow C, C \rightarrow A, C \rightarrow D, D \rightarrow A\}$ ，初始化 $PR(0)=0.25$ ，阻尼因子 $d=0.85$ ，收敛阈值 $\epsilon=10^{-6}$ 。

对于C节点，其第一轮的计算结果为：

$$PR^{(1)}(C) = d \left(\frac{PR^{(0)}(A)}{2} + \frac{PR^{(0)}(B)}{1} \right) + \frac{1-d}{4} = 0.85 \left(\frac{0.25}{2} + 0.25 \right) + 0.0375 = 0.319;$$

而其他节点同样按照上述计算形式。

经过 12 轮迭代，最大差值为 $\|PR^{(12)} - PR^{(11)}\|_1 = 8.3 \times 10^{-7} < \epsilon$ ，此时停止收敛，最终结果为如下表格：

节点	PR(0)	PR(12)
A	0.25	0.276
B	0.25	0.155
C	0.25	0.286
D	0.25	0.281

节点C因被多个高权重节点直接链接且自身出链分散（指向A、D），在阻尼因子作用下积累较高PageRank值；节点D通过闭环反馈（ $C \rightarrow D \rightarrow A \rightarrow C$ ）实现权重累积，而节点B因单一入链及低出度贡献成为网络中最不重要的节点，此分布体现了链接拓扑结构与随机跳转机制对节点重要性的联合影响。

2.2 具体实现

2.2.1 初始化与数据载入

在初始化阶段，构造函数接收并存储了PageRank类的核心参数，包括**阻尼系数**、**收敛阈值**、**最大迭代次数**和**结果截断数量**。这些参数的设定直接影响了后续迭代的收敛速度与结果精度，例如阻尼系数0.85是经典值，用于平衡随机跳转与正常转移的概率，而 $1e-8$ 的收敛阈值则确保了结果的高精度要求。

```
1 //PageRank类
2 double damping_; // 阻尼因子
3 double epsilon_; // 收敛阈值
4 int max_iter_; // 最大迭代次数
5 int top_k_; // 结果截断数量
6 //main函数
7 PageRank pr(0.85, 1e-8, 100, 100);
```

数据加载流程通过 `readEdges` 函数实现，其核心目标是构建图结构的内部表示。代码采用逐行读取边数据的方式，并通过集合 `edges_` 主动过滤重复边，避免了重复计算对图结构的影响。对于每条有效边，系统会同步更新三个关键数据结构：`in_links_` 记录每个节点的入边来源，`out_degree_` 统计节点的出度，`nodes_` 集合维护所有节点的唯一性。这种设计使得后续迭代阶段能够快速访问节点的入链关系和出度信息，为高效计算转移概率矩阵提供了基础。

```
1 void PageRank::readEdges(const std::string& filename) {
2     std::ifstream fin(filename);
3     if (!fin) {
4         throw std::runtime_error("Failed to open file: " + filename);
5     }
6
7     int u, v;
8     while (fin >> u >> v) {
9         auto edge = std::make_pair(u, v);
10        if (edges_.find(edge) == edges_.end()) {
11            edges_.insert(edge);
12            in_links_[v].push_back(u);
13            out_degree_[u]++;
14            nodes_.insert(u);
15            nodes_.insert(v);
16        }
17    }
18    fin.close();
19 }
```

从空间复杂度分析，主要数据结构的存储开销呈现线性特征。集合 `edges_` 和映射 `in_links_` 的空间复杂度均为 $O(E)$ ，其中 E 为边数，前者存储所有边的唯一性标识，后者按目标节点组织入边列表。映射 `out_degree_` 和集合 `nodes_` 的空间复杂度为 $O(N)$ ， N 为节点数，分别用于记录节点出度和维护节点全集。因此整体空间复杂度为 $O(E + N)$ ，属于典型的稀疏图存储方式，能够有效处理大规模网络数据。

由于输入数据没有重复边，若进一步优化，可通过牺牲部分代码简洁性来消除 `edges_` 集合的存储，从而将空间复杂度降低，但当前实现通过冗余存储保证了数据完整性的严格校验；在后续实验结果分析中为达到内存最优，可将数据重复边检验的流程除去。

2.2.2 数据预处理

在预处理阶段，`handleDeadEnds` 函数通过虚拟连接消除死胡同节点对计算的影响：当节点出度为0时，将其出度设为全图节点数 N ，并建立该节点到所有节点的虚拟边。这种处理将死胡同节点的转移概率均摊到全图节点，避免概率泄漏问题，同时保证转移矩阵的随机性，其时间复杂度为 $O(DN)$ （ D 为悬挂节点数），空间开销增加 $O(DN)$ 的虚拟边存储。

```
1 void PageRank::handleDeadEnds() {
2     const int N = nodes_.size();
3     for (const auto& node : nodes_) {
4         if (out_degree_[node] == 0) {
5             out_degree_[node] = N;
6             for (const auto& neighbor : nodes_) {
7                 in_links_[neighbor].push_back(node);
8             }
9         }
10    }
11 }
```

⚠ Caution

值得注意的是，虚拟边的引入可能导致边数 E 膨胀至 $O(N^2)$ 的最坏情况（当多数节点为悬挂节点时），此次数据集中死胡同节点占比高达10%，导致后续计算的空间和时间复杂度大大增大，因此在实验结果分析中可恰当的选择是否对死胡同节点进行单独处理。

2.2.3 PageRank计算

PageRank计算采用迭代逼近法实现核心算法，其逻辑架构围绕概率转移、收敛控制与数值稳定性展开。

计算核心 `calculate` 函数通过幂迭代法求解平稳分布。初始化阶段为所有节点赋予均匀概率 $1/N$ ，时间复杂度 $O(N)$ 。迭代过程中，每个节点的新PageRank值由入链节点的贡献加权求和，叠加阻尼系数 $(1-d)/N$ 的随机跳转概率。

其中入链贡献计算通过遍历 `in_links_` 结构获取源节点，结合其出度进行概率分配，单次迭代时间复杂度为 $O(E+N)$ （ E 为有效入边和）。收敛判定采用 $L1$ 范数差值，当变化量小于阈值或达到最大迭代次数时终止，整体时间复杂度为 $O(K(E+N))$ （ K 为实际迭代次数）。空间复杂度主要来自存储入边列表 $O(E)$ 、节点出度 $O(N)$ 和两个PageRank向量 $O(N)$ ，总空间复杂度为 $O(E+N)$ 。

```
1 void PageRank::calculate() {
2     initializeRank();
3     std::unordered_map<int, double> pr_new;
4
5     for (int iter = 0; iter < max_iter_; ++iter) {
6         double diff = 0.0;
```



```

7         pr_new.clear();
8
9         // 计算新的PageRank值
10        for (const auto& node : nodes_) {
11            double sum_in = 0.0;
12            for (const auto& src : in_links_.at(node)) {
13                sum_in += pr_[src] / out_degree_.at(src);
14            }
15            pr_new[node] = (1.0 - damping_) / nodes_.size() + damping_ *
sum_in;
16        }
17
18        // 计算差值并更新
19        for (const auto& node : nodes_) {
20            diff += std::fabs(pr_new[node] - pr_[node]);
21            pr_[node] = pr_new[node];
22        }
23
24        // 收敛检查
25        if (diff < epsilon_) {
26            std::cerr << "Converged at iteration: " << iter + 1
27                << " (diff=" << diff << ")\n";
28            break;
29        }
30    }
31    normalize();
32 }

```

收敛性通过计算两次迭代间所有节点权值的绝对差之和 $diff$ 进行判定，当 $diff < \epsilon$ 时提前终止迭代，否则在达到最大迭代次数后强制停止，最后通过 `normalize` 函数通过概率归一化消除浮点误差累积，确保结果总和为1，其时间复杂度 $O(N)$ 可忽略不计。上述实现通过哈希表存储中间结果，牺牲部分缓存局部性换取动态扩容能力，适合处理非连续节点编号的大规模稀疏网络。

3. BLOCK-STRIPES

3.1 算法概述

3.1.1 初始版本中的问题

- **数据结构导致的内存消耗问题：**初始版本中使用如下数据结构

```

1 unordered_map<int, vector<int>> in_links; // 记录每个节点的入链表
2 unordered_map<int, int> out_degree;      // 记录每个节点的出度
3 unordered_set<int> nodes;                // 所有出现过的节点集合

```

unordered_map和vector的组合在存储和访问时会有比较大的开销，且map键值存储需要进行额外的哈希计算，效率很低。

- **对边进行去重时的内存开销**

由于我们不能保证数据集中的边数据不会重复，所以需要在读取数据的时候对重复的边进行去重，而初始版本的算法中使用unordered_set来存储边，但是其中进行的哈希运算会引入额外的内存和运算开销

- **对于死胡同节点的处理操作**

初始版本直接将出度为0的节点显式的连接到所有节点，然而这样会使得邻接表的内存占用大幅度增加，如果图的规模过大，甚至会出现内存不足的情况

- **计算时的不连续访问及遍历开销**

初始版本计算的代码如下：

```
1  for (int node : nodes) {
2      double sum_in = 0.0;
3      for (int src : in_links[node]) {
4          sum_in += pr[src] / out_degree[src];
5      }
6      pr_new[node] = (1.0 - DAMPING) / N + DAMPING * sum_in;
7  }
```

此处遍历邻接表时，访问的模式是不连续的，这会导致缓存的命中率极低，导致额外的时间开销。

且每次迭代都要遍历所有节点和边，效率极低。

综上所述，我们使用CSR稀疏矩阵以及块矩阵优化方式来重新设计算法

3.1.2 CSR稀疏矩阵格式

CSR（Compressed Sparse Row，压缩稀疏行）是一种高效存储稀疏矩阵的数据结构，CSR 格式通过只存储非零元素及其位置，避免了存储大量的零值，从而节省内存并提高计算效率。

例子如下：

对于如下矩阵

```
1  0  5  0  0
2  3  0  0  0
3  0  0  0  7
4  0  0  1  0
```

- 如果使用二维数组来存储，就需要存储所有16个元素，其中有12个0
- 而使用稀疏矩阵只需要存储四个非零元素和行列位置，节省了大量空间

CSR 格式将稀疏矩阵分为三个部分存储：

1. **values** :
 - 存储所有非零元素的值，按行的顺序依次存储。
 - 例如，上述矩阵的非零元素是 `[5, 3, 7, 1]`。
2. **col_idx** (**列索引**) :
 - 存储每个非零元素所在的列号。
 - 例如，上述矩阵中非零元素的列号是 `[1, 0, 3, 2]`。
3. **row_ptr** (**行指针**) :
 - 存储每一行的非零元素在 `values` 数组中的起始位置。
 - 例如：

- 第 0 行的非零元素从 `values[1]` 开始。
 - 第 1 行的非零元素从 `values[0]` 开始。
 - 第 2 行的非零元素从 `values[3]` 开始。
 - 第 3 行的非零元素从 `values[2]` 开始。
- 对应的 `row_ptr` 是 `[1, 0, 3, 2, 4]`（注意：最后一个值是非零元素的总数，用于计算范围）。

3.1.3 块矩阵优化（分块计算）

在现实应用中，pagerank计算的数据集往往很大，转移矩阵的维度为N×N，对于大规模的图（例如数十亿节点），矩阵无法直接载入到内存中进行计算，并且原始的计算方案会引入许多从内存读取数据到cache乃至从硬盘读取数据到内存的额外开销，导致算法的效率极低，而且原始的计算方案会限制分布式计算资源的使用，因此采用分块的计算方案来优化pagerank算法非常重要。

3.1.4 分块矩阵的核心思想

将转移矩阵M和节点集划分为多个逻辑块，每一个块都对应了一个子矩阵和子向量，从而计算可分块独立进行，最后合并结果

分块矩阵格式如下：

$$M = \begin{bmatrix} B_{11} & B_{12} & \cdots & B_{1K} \\ B_{21} & B_{22} & \cdots & B_{2K} \\ \vdots & \vdots & \ddots & \vdots \\ B_{K1} & B_{K2} & \cdots & B_{KK} \end{bmatrix}, \quad \mathbf{pr} = \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \vdots \\ \mathbf{v}_K \end{bmatrix}$$

矩阵划分和子块定义：

将节点集划分为K个子集，并构建子矩阵，每个子矩阵B_{ij}包含从块i到块j的所有边，若块i有n_i个节点，块j有n_j个节点，则B_{ij}的维度为n_j×n_i。

假设节点分为3块（K=3），则：

$$M = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{bmatrix}$$

块 B₁₂ 存储从块1节点到块2节点的所有转移概率。

分块后的PageRank迭代公式

将全局公式分解为块级计算：1_j 是块 j 的全1向量，每个块的更新仅依赖其他块的PageRank子向量 v_i(k)。

$$\mathbf{v}_j^{(k+1)} = \frac{1-d}{N} \mathbf{1}_j + d \sum_{i=1}^K B_{ji} \cdot \mathbf{v}_i^{(k)}$$

分块后，**局部性增强**：每个块的计算可独立进行，适合分布式并行，计算某个块的时候仅需加载当前块的子矩阵和关联的子向量。

那么综上所述，我们

3.1.5 优化设计

$$\mathbf{pr}^{(k+1)} = \frac{1-d}{N} \mathbf{1} + d \cdot \left(\sum_{i=1}^K B_{1i} \mathbf{v}_i^{(k)}, \sum_{i=1}^K B_{2i} \mathbf{v}_i^{(k)}, \dots, \sum_{i=1}^K B_{Ki} \mathbf{v}_i^{(k)} \right)^T$$

$$n^2 \cdot \text{sizeof(double)} \leq M$$

$$P = \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right]$$

3.2 具体实现

4. 参考文献

1. Brin, S., & Page, L. (1998). *The Anatomy of a Large-Scale Hypertextual Web Search Engine*. Computer Networks and ISDN Systems, 30(1-7), 107-117.
2. Langville, A. N., & Meyer, C. D. (2006). *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press.
3. Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). *The PageRank Citation Ranking: Bringing Order to the Web*. Stanford InfoLab.
4. Haveliwala, T. H. (2002). *Topic-Sensitive PageRank*. Proceedings of the 11th International Conference on World Wide Web, 517-526.
5. Gyongyi, Z., Garcia-Molina, H., & Pedersen, J. (2004). *Combating Web Spam with TrustRank*. Proceedings of the 30th International Conference on Very Large Data Bases, 576-587.